

Project01

컴퓨터소프트웨어학부 2022095287 김준서

Setting

- 실행 환경: Ubuntu-22.04

Design

sys_getpid

- 부모 프로세스의 pid를 반환하는 함수
- `syscall.h`: `getpid` 시스템 콜 번호 상수(`SYS_getpid`) 정의
- `syscall.c`: `sys_getpid` 함수 선언 / `syscalls` 함수 포인터 배열에 `sys_getpid` 함수 추가
- `sysproc.c`: `sys_getpid` 함수 정의

getpid

- 인터럽트를 발생시켜 `sys_getpid` 함수를 실행시키는 함수
- `user.h`: `getpid` 함수 선언 / 유저 프로그램용 헤더
- `usys.S`: `getpid` 함수 정의

project01 (user program)

- `project01.c`
 - `getpid`, `getgid` 시스템 콜을 이용해 pid, gid를 출력하는 프로그램
 - `user.h`를 include해서 `getpid`, `getgid` 함수 사용
- `Makefile`: `UPROGS`와 `EXTRA`에 `_project01` 추가

Implementation

sys_getpid

- `syscall.h`: 매크로를 사용하여 `getpid` 시스템 콜 번호를 `SYS_getpid`(=22)로 정의

```
20 20 #define SYS_link 19
21 21 #define SYS_mkdir 20
22 22 #define SYS_close 21
23 + #define SYS_getpid 22
```

- `syscall.c`: `sys_getpid` 함수 선언 / `syscalls[SYS_getpid]`를 `sys_getpid`로 초기화

```
105 105 extern int sys_uptime(void);
106 + extern int sys_getpid(void);
```

```

128 | 129 | [SYS_close] sys_close,
    | 130 | + [SYS_getgid] sys_getgid,
129 | 131 | };

```

- `sysproc.c`: 부모 프로세스가 존재하지 않을 경우 -1을 반환 (`init` 프로세스가 있으므로, 정상적인 경우 부모 프로세스는 항상 존재)

- `proc.c`의 `myproc` 함수: 현재 프로세스 정보를 담은 `proc` 구조체를 반환
- `proc` 구조체의 멤버
 - `struct proc *parent`: 부모 프로세스 정보를 담은 구조체 변수의 주소
 - `int pid`: 프로세스 id

```

44 | 44 |
    | 45 | + int
    | 46 | + sys_getgid(void)
    | 47 | + {
    | 48 | +     struct proc* parent = myproc()->parent;
    | 49 | +     if(parent == 0) {
    | 50 | +         return -1;
    | 51 | +     }
    | 52 | +     return parent->pid;
    | 53 | + }
    | 54 | +

```

getgid

- `user.h`: `getgid` 함수 선언

```

24 | 24 | int sleep(int);
25 | 25 | int uptime(void);
    | 26 | + int getgid(void);

```

- `usys.s`: `getgid` 함수 정의

- `SYSCALL(getgid)`: `SYS_getgid`를 `%eax` 레지스터에 저장하고 인터럽트 발생시킴

```

30 | 30 | SYSCALL(sleep)
31 | 31 | SYSCALL(uptime)
    | 32 | + SYSCALL(getgid)

```

project01 (user program)

- `project01.c`

```

1 | + #include "types.h"
2 | + #include "user.h"
3 | +
4 | + int
5 | + main(int argc, char *argv[])
6 | + {
7 | +     const char* sid = "2022095287";
8 | +     printf(1, "My student id is %s\n", sid);
9 | +     printf(1, "My pid is %d\n", getpid());
10 | +     printf(1, "My gid is %d\n", getgid());
11 | +     exit();
12 | + }

```

- `Makefile`

- `UPROGS`와 `EXTRA`에 `_project01` 추가
- 유저 프로그램 `x.c`는 빌드 후 `_x` 실행파일이 됨

182	182		_wc\
183	183		_zombie\
	184	+	_project01\
254	255		README dot-bochsrc *.pl toc.* runoff runoff1 runoff.list\
255	256		.gdbinit.tmpl gdbutil\
	257	+	_project01\

Result

- 빌드 커맨드: `chmod +x sign.pl; chmod +x vectors.pl; chmod +x bootxv6.sh; make clean; make; make fs.img; ./bootxv6.sh`
- xv6를 실행한 후, 미리 작성되어 있는 `project01` 유저 프로그램을 실행
 - `project01` 유저 프로그램에서 `getpid`, `getgid` 시스템 콜을 사용해 결과를 출력하는 모습을 확인할 수 있음

```
SeaBIOS (version 1.15.0-1)

ipXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B4A0+1FECB4A0 CA00

Booting from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ project01
My student id is 2022095287
My pid is 3
My gpid is 2
$
```

Trouble Shooting

1. `sys_getpid` 함수를 wrapper 함수로 구현할 것인가?
 - `sysproc.c`의 프로세스 관련 시스템 콜들의 구현 방식을 분석
 - `sys_fork`, `sys_exit`, `sys_wait` 등: `proc.c`의 `fork`, `exit`, `wait` 함수를 호출하는 wrapper 함수의 형태
 - `sys_getpid`: 직접 pid를 구해서 리턴하는 형태
 - `getgid` 시스템 콜의 경우 로직이 간단하고 `getpid`와 유사하므로 `sysproc.c`에서 바로 구현하기로 결정
2. `defs.h`에 `getgid` 함수를 선언해야 하는가? (OS단에서 `getgid` 함수가 필요한가?)
 - `defs.h`: OS 구현 과정에서 사용할 구조체/함수/매크로를 모아 선언해놓은 헤더 파일
 - `defs.h`와 `user.h`에 선언된 시스템 콜 함수들은 이름은 같아도 내용이 다름
 - ex) `fork`
 - `defs.h`의 `fork`: `proc.c`에 정의된, 핵심 기능을 수행하는 함수
 - `user.h`의 `fork`: `usys.s`에 정의된, 인터럽트를 발생시키는 함수 (유저 프로그램은 `usys.o`와 같이 빌드됨)

- 즉, `defs.h`에 `getgpid` 함수를 추가하려면 `sys_getgpid` 함수를 wrapper 함수 형태로 바꿔야 함 (1번 결정사항을 반복해야 함)
- 기존 구현을 굳이 변경할 필요성을 느끼지 못하여, `getpid`와 마찬가지로 `getgpid` 함수 또한 `defs.h`에 선언하지 않음